VISIONASSIST: An AI-Based Real-Time Scene Narration System for the Visually Impaired

A Project report submitted in partial fulfillment of the requirements for the award of degree in

BACHELOR OF TECHNOLOGY

(COMPUTER SCIENCE AND ENGINEERING) SUBMITTED BY

Registration number	Name of the student
A22126510209	G. Likitha Sri
A22126510169	P. Manasa
A22126510159	L. Pavan Sai
A22126510162	M. Srinivasa Kalyan

UNDER THE GUIDANCE OF

Dr. Ch. Lakshmi Narayana (M.Tech, Ph.D)

Assistant professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINNERING

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES (A+) SANGIVALASA, VISAKHAPATNAM – 531162

January - 2026

S.NO	CONTENTS
1	Introduction
2	System Architecture
3	Modules Division
4	DFD'S &UML Diagram
5	Source Code
6	Outputs Screens
7	Conclusion
8	References

INTRODUCTION

Blind and visually impaired people face significant challenges in understanding their surroundings every day. Even ordinary situations, like walking outside or crossing a road, become difficult when information about the environment is missing. Modern technology, especially artificial intelligence, is making it easier for blind people to navigate the world with confidence. Assistive devices and AI-powered systems can help detect objects, recognize people, and even describe actions and scenes using voice output. By using a camera and smart algorithms, these solutions convert visual information into spoken words that blind users can understand easily.

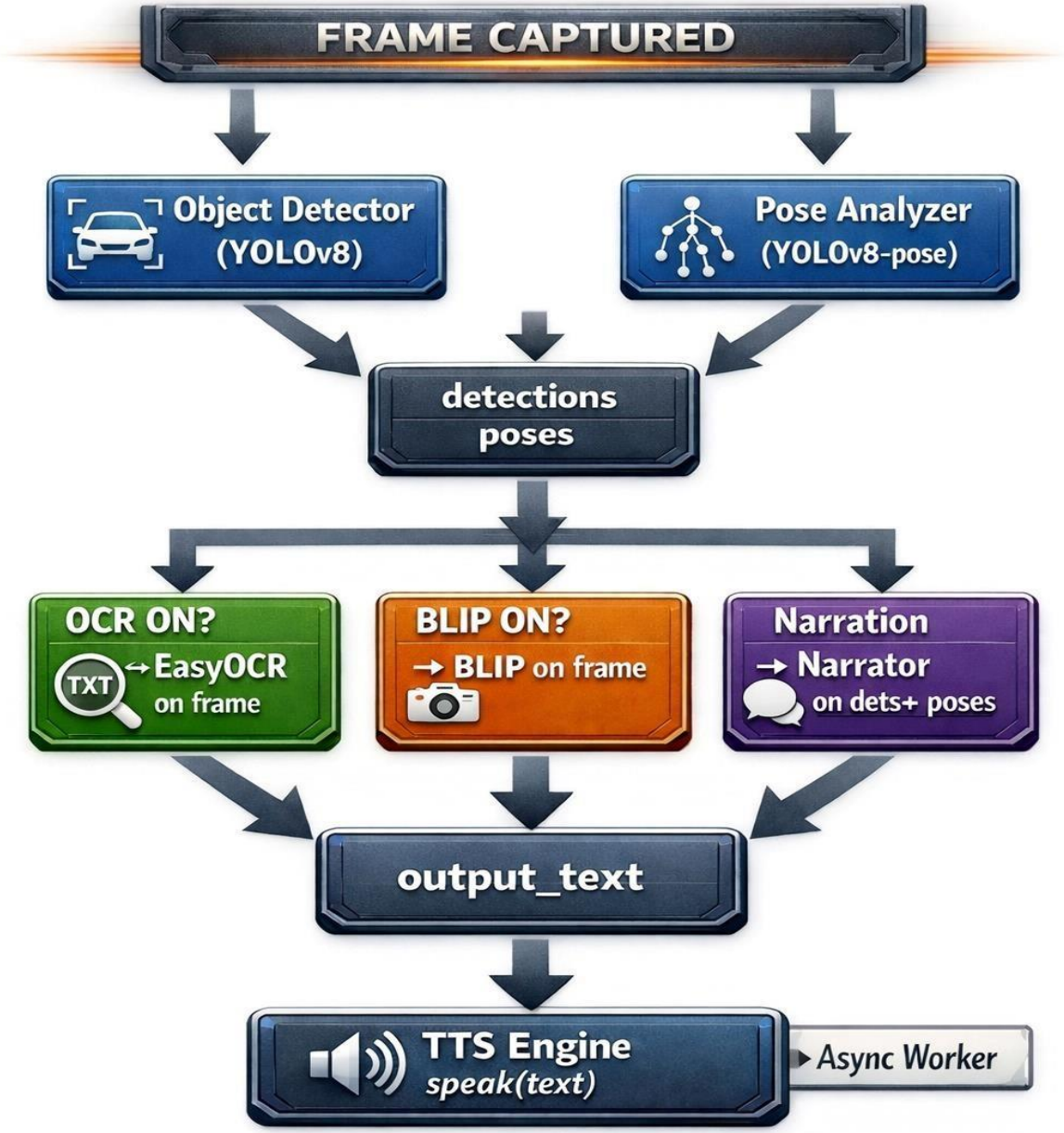
This project aims to develop a smart assistive technology system for blind people that can recognize objects and actions in real-time, then narrate what is happening through voice. For example, if a person is sitting and drinking water, the system will describe it as "A person is sitting and drinking water." With such a tool, blind users gain independence, safety, and greater participation in daily life. This approach combines the latest advances in computer vision (like YOLO object detection, pose estimation) and natural language processing to offer practical, accessible solutions for those who cannot rely on sight.

-

Key Points

- **Assistive technology** helps blind people become more independent by converting visual scenes into spoken descriptions.
- **AI models** like YOLO can detect objects (people, chairs, bottles) in the environment in real-time.
- Adding **action recognition** or pose analysis helps the system describe what people are doing (e.g., sitting, drinking, walking).
- **Voice output** delivers this scene information in a way that is easy for blind users to understand and use in their daily lives.
- Such technology can be implemented on simple devices or smartphones, making it widely accessible.
- Benefits include improved mobility, safety (e.g., knowing if vehicles or obstacles are ahead), and confidence in social and public settings.
- The project leverages existing datasets and models to create a user-friendly, real-time assistive system for the visually impaired.

SYSTEM ARCHITECTURE



MODULES DIVISION

Module Name	Module Objective	Core Responsibilities	Technologies / Libraries Used
Camera Acquisition Module	Capture live visual data of the user's surroundings	Continuous webcam frame capture, frame resizing and preprocessing	OpenCV, System Camera
Object Detection Module	Identify and localize objects in real time	Detect people, vehicles, daily-use objects; estimate distance to obstacles	YOLOv11, Ultralytics, PyTorch
Pose Estimation & Action Analysis Module	Analyze human posture and infer actions	Detect body keypoints; classify actions such as standing, sitting, walking	MediaPipe Pose
Scene Understanding Module	Understand overall scene context	Combine object detection, pose data, and spatial reasoning	Custom rule-based logic
Scene Captioning Module (New)	Generate highlevel semantic descriptions	Produce natural language captions for entire scenes	BLIP (Bootstrapped Language–Image Pretraining), HuggingFace Transformers
Text Recognition Module (OCR)	Read visible text in the environment	Detect and recognize printed text from images	EasyOCR, PyTorch
Narration Control Module	Decide what information should be spoken	Prioritize safety alerts, scene narration, OCR, or captions	Custom control logic
Voice Output Module	Convert text descriptions into speech	Text-to-speech synthesis and audio playback	pyttsx3 (Offline TTS)
User Interaction Module	Enable user control over system modes	Toggle narration, OCR mode, captioning mode, mute/unmute	Keyboard-based controls

METHODOLOGY (Algorithms and Workflow)

1. Real-Time Frame Acquisition (Camera Streaming)

- The system continuously captures live frames from the camera (no voice trigger required).
- Each frame is resized/preprocessed for faster inference.
- Frames are processed in a continuous loop until the user quits.

Workflow:

1. Open camera stream (OpenCV).
2. Read frame continuously.
3. Preprocess frame (resize/format).
4. Send frame into AI pipeline.

---

2. Object Detection + Distance Estimation (YOLOv11 + Monocular Depth Logic)

- YOLOv11 is used for real-time object detection in every frame. • For each detected object, bounding box and confidence score are produced.
- Distance estimation is computed using monocular pinhole camera approximation:

Distance=focal_length×real_heightpixel_heightDistance = \frac{focal_length \times real_height}{pixel_height}Distance=pixel_heightfocal_length×real_height

- Each object is assigned a safety level based on distance thresholds:
- Safe
- Warning
- Danger **Workflow**:

1. Input frame to YOLO model.
2. Detect objects → bbox, class, confidence.
3. Measure object pixel height from bbox.
4. Estimate distance (meters).
5. Assign safety level based on distance.

---

3. Human Pose Detection and Action Recognition (YOLOv11 Pose / PoseAnalyzer) • If enabled, the system detects human pose keypoints in each frame.

- Pose landmarks are used to infer human posture and actions such as:
- standing
- walking
- sitting
- Actions are attached to tracked persons using track ID.

Workflow:

1. Input frame into pose model.
2. Extract bounding box + keypoints.
3. Compute posture/action label.

4. Attach action output to the corresponding person.

---

4. Scene Understanding and Object–Action Association

- Scene understanding combines outputs of:
- detected objects
- distances + safety level
- human pose/action recognition
- The system prioritizes useful narration instead of reading a list.
- Objects/actions are filtered using thresholds to reduce noise:
- max objects per narration
- action cooldown
- repeat suppression **Workflow**:

1. Collect objects + pose results.
2. Link person actions with nearby objects when required.
3. Filter low-priority details.
4. Prepare structured scene summary.

---

5. **Multi-Mode Information Extraction (Scene / OCR / BLIP)** The project now works in multiple modes, controlled by the user: a) Scene Narration Mode (Default)

• Generates real-time narration based on detection + pose + distances.

b) OCR Text Reading Mode (Toggle key: o)

- OCR reads text from the camera feed (useful for signs, posters, labels).
- Center-crop + preprocessing improves OCR speed.
- OCR output is narrated only when meaningful text is detected.

c) BLIP Caption Mode (Toggle key: b)

- BLIP generates a natural-language caption of the entire frame:
- “a person sitting at a desk”
- Useful for high-level environment description.

Workflow:

1. Check active mode (Scene / OCR / BLIP).
2. Run the respective inference module.
3. Generate output text.
4. Send text to TTS.

---

6. Narration Generation (SceneNarrator Logic)

- The narration is generated using rule-based formatting and prioritization:
- distance-aware descriptions (“person ahead at 1.5 meters”)
- action-aware descriptions (“person is walking”)
- object-aware descriptions (“chair on the left”)
- Narration avoids repeating the same sentence frequently.

Workflow:

1. Convert detections into natural phrases.
2. Prioritize danger-level warnings.
3. Combine short sentences into one narration output.
4. Suppress repeats using time-based cooldown.

---

7. Speech Output (TTS Engine with Priority Queue)

- Text output is converted to speech using TTS engine.
- System supports priority-based speech:
- High priority: danger/warning alerts (interrupt previous queue)
- Normal priority: general narration
- This prevents delayed narration and ensures user safety.

Workflow:

1. Output text produced (Scene/OCR/BLIP).
2. Determine priority level (danger = high).
3. Clear pending queue if necessary.
4. Speak immediately through audio output.

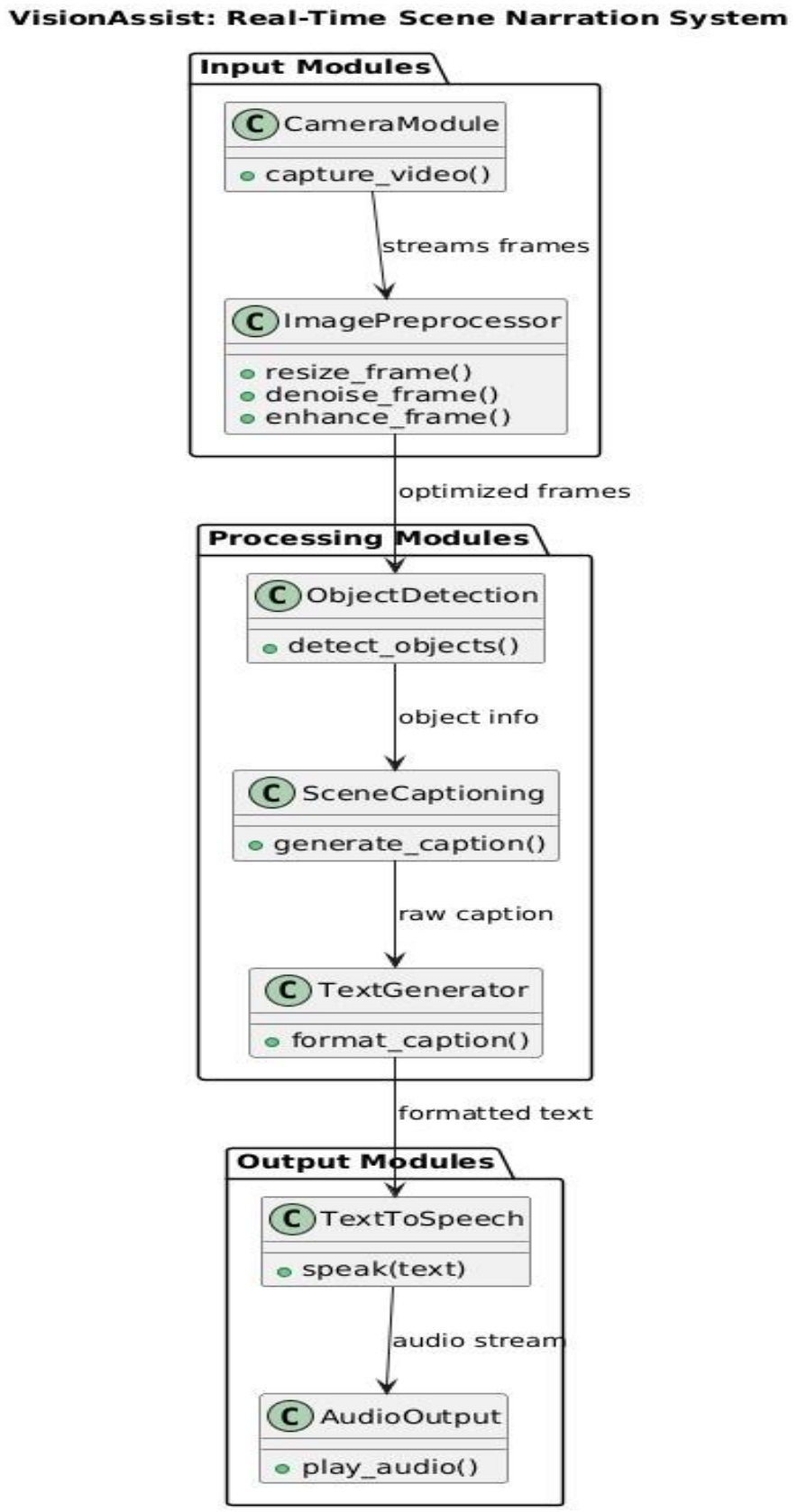
---

8. User Interaction and Controls (Keyboard Controls / Real-time Switching) The system supports blind-friendly interaction using keyboard shortcuts:

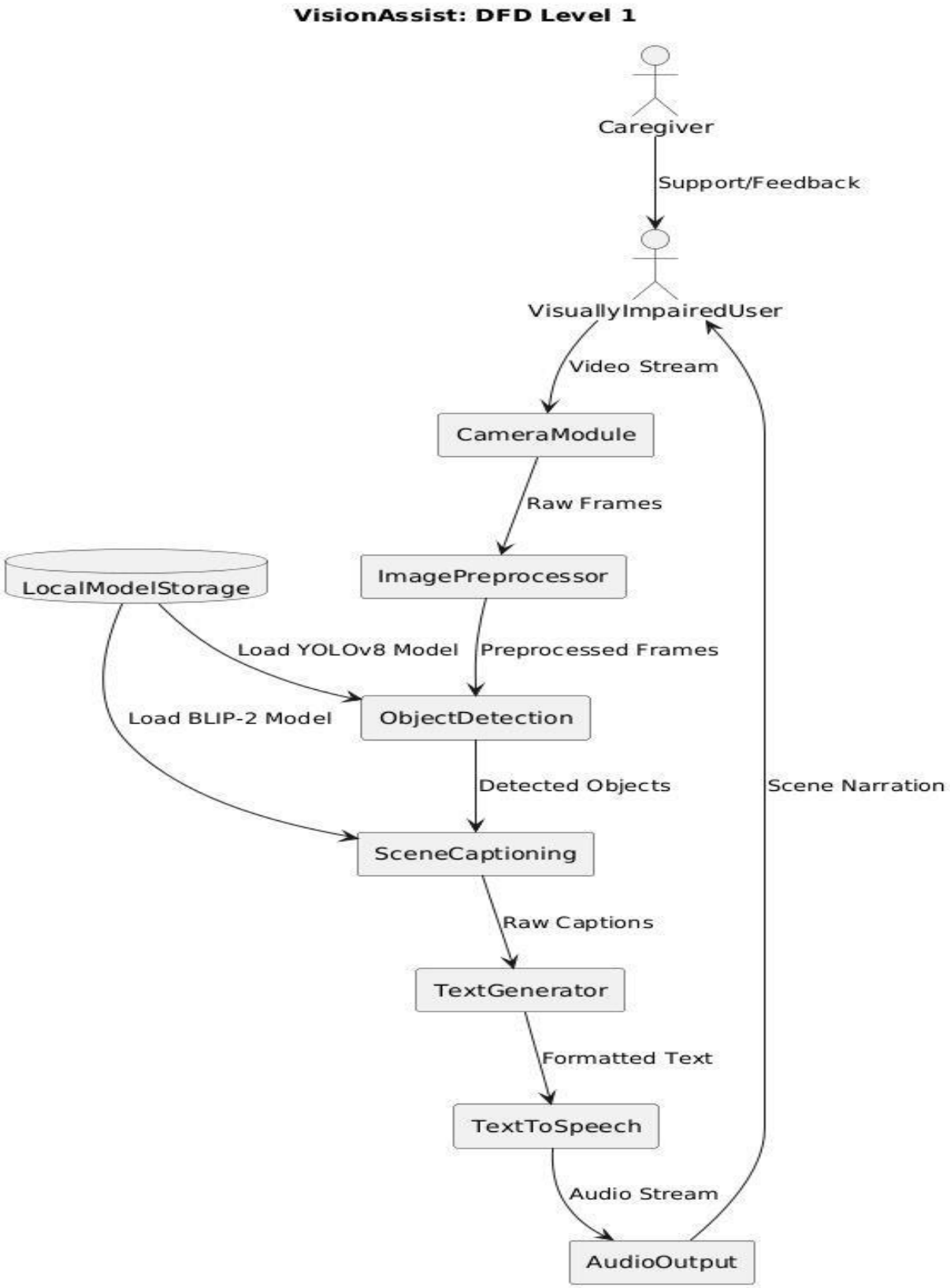
- **S-> Start/Pause narration**
- **M->Mute/Unmute voice**
- **O->OCR toggle**
- **B->BLIP toggle**
- **Q->Quit System Workflow**:

1. Capture key press from OpenCV window.
2. Toggle mode/state.
3. Provide immediate audio feedback (“OCR enabled”).

UML DIAGRAM


VisionAssist: Real-Time Scene Narration System
Input Modules
CameraModule
capture_video()
streams frames
ImagePreprocessor
resize_frame()
denoise_frame()
enhance_frame()
optimized frames
Processing Modules
ObjectDetection
detect_objects()
object info
SceneCaptioning
generate_caption()
raw caption
TextGenerator
format_caption()
formatted text
Output Modules
TextToSpeech
speak(text)
audio stream
AudioOutput
play_audio()

DFD DIAGRAM


VisionAssist: DFD Level 1
Caregiver
Support/Feedback
VisuallyImpairedUser
Video Stream
CameraModule
Raw Frames
LocalModelStorage
ImagePreprocessor
Load YOLOv8 Model
Preprocessed Frames
Load BLIP-2 Model
ObjectDetection
Detected Objects
Scene Narration
SceneCaptioning
Raw Captions
TextGenerator
Formatted Text
TextToSpeech
Audio Stream
AudioOutput

DATASETS

In this project, we **do not directly download or train on the COCO dataset**. Instead, we use the **Ultralytics YOLOv11 pretrained model (yolov11n.pt)**, which has already been trained on the **COCO dataset**.

- **COCO** is a widely used large-scale dataset for: o Object detection o Segmentation o Human keypoints o Image captioning
- It contains:
  - ~330,000 images
  - ~200,000+ labeled/annotated images o **80 common object categories** such as: person, car, chair, bottle, cellphone, laptop, etc.
- Since the model is pretrained, the system is able to detect COCO classes **without requiring dataset training inside our project**.
- The project performs object detection using **YOLOv11 pretrained weights**
- The pretrained weights are based on **COCO dataset categories**
- We are using **real-time webcam input** for inference (not COCO dataset images)

SOURCE CODE

main.py

```
"""
Main entry point for Scene Narration System
"""

import argparse import sys from
main_pipeline import MainPipeline from
config import Config

def main():
    """Main entry point."""    parser = argparse.ArgumentParser(    description='Scene
Narration System - Real-time object detection, pose estimation,
and voice narration'
    )
```

```
    parser.add_argument('--camera', '-c', type=int, default=Config.CAMERA_INDEX,
help='Camera device index')
                                                parser.add_argument('--display',  '-d',
action='store_true',
default=Config.DISPLAY_VIDEO,
                    help='Display video feed')     parser.add_argument('--no-
pose', action='store_true',                    help='Disable pose detection')
                     parser.add_argument('--confidence',         type=float,
default=Config.DETECTION_CONFIDENCE,
                    help='Detection confidence threshold')
parser.add_argument('--config', action='store_true',
help='Print configuration and exit')

    args = parser.parse_args()

    if args.config:
        print("=" * 60)        print("Scene Narration
System - Configuration")        print("=" * 60)
config_dict = Config.to_dict()        for key, value in
config_dict.items():
            print(f"{key}: {value}")
print("=" * 60)        return

    # Build configuration     config =
Config.to_dict()     config['camera_index'] =
args.camera     config['display_video'] =
args.display     config['use_pose_detection'] = not
args.no_pose     config['detection_confidence'] =
args.confidence

    # Print welcome message     print("=" * 60)
print("Scene Narration System")     print("=" * 60)
print("Features:")     print("  • Real-time object detection
(YOLO)")     print("  • Distance estimation")     print("  •
Human pose detection and action recognition")     print("  •
Object-action association")
```

```
    print("  • Natural language scene narration")
print("  • Voice output (TTS)")     print("  •
Safety alerts")     print("=" * 60)     print()

    try:
        # Create and run pipeline
pipeline = MainPipeline(config)
pipeline.run()

    except KeyboardInterrupt:
print("\nShutting down...")        sys.exit(0)

    except Exception as e:
print(f"\nError: {e}")        import
traceback
traceback.print_exc()
sys.exit(1)

if __name__ == '__main__':
main()


mainpipeline.py
"""
Main processing pipeline that coordinates all components
"""

import cv2 import
numpy as np import
time from typing
import Dict

from ocr_reader import OCRReader from
camera_capture import CameraCapture from
object_detector import ObjectDetector from
pose_analyzer import PoseAnalyzer from
scene_narrator import SceneNarrator from
tts_engine import TTSEngine from
user_controls import UserControls
```

```
from blip_captioner import BLIPCaptioner

class MainPipeline:
    """Main pipeline coordinating all components."""

    def __init__(self, config: Dict):
        self.config = config

        print("Initializing Scene Narration System...")

        # Camera        self.camera =
CameraCapture(
camera_index=config["camera_index"],
width=config["camera_width"],
height=config["camera_height"],
url=config.get("camera_url")
        )

        # Object detector        self.object_detector =
ObjectDetector(
model_path=config["yolo_model"],
confidence=config["detection_confidence"],
camera_fov=config["camera_fov"],
frame_height=config["camera_height"]
        )

        # Pose analyzer        self.pose_analyzer =
None        if config.get("use_pose_detection",
True):            try:
                self.pose_analyzer = PoseAnalyzer(
model_path=config["pose_model"],
confidence=config["detection_confidence"]
                )
                print("Pose detection enabled")
except Exception as e:
                print(f"Warning: Could not initialize pose detection: {e}")
self.pose_analyzer = None

        # Scene narrator        self.scene_narrator =
SceneNarrator(
narration_interval=config["narration_interval"],
```

```
        repeat_suppression=2.5,
max_objects=2,
action_cooldown=2.0
    )

    # TTS engine
self.tts_engine = TTSEngine(
rate=config["tts_rate"],
volume=config["tts_volume"]
    )

    # User controls      self.user_controls
=                          UserControls()
self.user_controls.set_callbacks(
on_start=self._on_start,
on_pause=self._on_pause,
on_mute=self.tts_engine.mute,
on_unmute=self.tts_engine.unmute
    )

    # OCR Reader (EasyOCR)
self.ocr_reader = None      if
config.get("ocr_enabled", True):
try:
        self.ocr_reader = OCRReader(
lang=config.get("ocr_lang", ["en"]),
interval=config.get("ocr_interval", 2.0),
crop_ratio=config.get("ocr_center_crop", 0.6),
conf_thres=config.get("ocr_conf_thres", 0.5),
        )
        print("OCR module loaded (Press 'o' to toggle)")
except Exception as e:
        print(f"Warning: OCR init failed: {e}")
self.ocr_reader = None
    # BLIP Captioner
self.blip_captioner = None      if
config.get("blip_enabled", True):
try:
        self.blip_captioner = BLIPCaptioner(
interval=config.get("blip_interval", 3.0)
        )
        print("BLIP captioning loaded (Press 'b' to toggle)")
```

```
        except Exception as e:
            print(f"Warning: BLIP init failed: {e}")
self.blip_captioner = None

    # Performance
self.frame_count = 0        self.fps =
0        self.last_fps_time =
time.time()

    print("System initialized successfully!")

  def _on_start(self):
    pass

  def _on_pause(self):
    self.tts_engine.stop()

  def process_frame(self, frame: np.ndarray) -> Dict:
    start_time = time.time()

    detections = self.object_detector.detect(frame)

    poses = []        if
self.pose_analyzer:
        poses = self.pose_analyzer.detect_poses(frame)

    output_text = None
mode = "scene"

    # OCR mode priority        if
self.user_controls.is_ocr_enabled() and self.ocr_reader:
        mode = "ocr"
try:
            text = self.ocr_reader.extract_text(frame)
if text:
                output_text = f"Text says: {text}"
self.tts_engine.speak(output_text, priority="high")            except
Exception as e:
            print(f"[OCR]        Runtime        crash:        {type(e).__name__}:        {e}")
self.tts_engine.speak("OCR failed. Switching back to narration.",
priority="high")
            self.user_controls.toggle_ocr()
```

```
            mode = "scene"
        #   BLIP caption mode (second priority)        elif
self.user_controls.is_blip_enabled() and self.blip_captioner:
            mode = "blip"            caption =
self.blip_captioner.generate_caption(frame)            if
caption:
            output_text = f"Scene: {caption}"
self.tts_engine.speak(output_text, priority="high")
        # narration mode        elif
self.user_controls.is_narration_enabled():
            output_text = self.scene_narrator.generate_narration(detections, poses)
if output_text:
            has_danger = any(d.get("safety_level") == "danger" for d in detections)
priority = "high" if has_danger else "normal"                self.tts_engine.speak(output_text,
priority=priority)

        # FPS        self.frame_count
+= 1        if self.frame_count % 30
== 0:
            now = time.time()            elapsed = now -
self.last_fps_time            self.fps = 30 / elapsed if
elapsed > 0 else 0            self.last_fps_time =
now

        processing_time = time.time() - start_time

        return {
            "detections": detections,
            "poses": poses,
            "output_text": output_text,
            "mode": mode,
            "processing_time": processing_time,
            "fps": self.fps,
            "frame_count": self.frame_count,
        }

    def draw_results(self, frame: np.ndarray, results: Dict):
        detections = results.get("detections", [])
poses = results.get("poses", [])

        # object boxes        if
self.config.get("show_bbox", True):
```

```
        for det in detections:
            x1, y1, x2, y2 = det["bbox"].astype(int)
cname = det["class_name"]              conf =
det["confidence"]              dist =
det.get("distance", 0)              safety =
det.get("safety_level", "safe")

            if safety == "danger":
                color = (0, 0, 255)
elif safety == "warning":
                color = (0, 165, 255)
else:
                color = (0, 255, 0)

            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)

            label = f"{cname} {conf:.2f}"              if
self.config.get("show_distance", True):
                label += f" {dist}m"              cv2.putText(frame,
label, (x1, y1 - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

    # pose boxes
for pose in poses:
        bbox = pose["bbox"].astype(int)          action = pose.get("action",
"unknown")          cv2.rectangle(frame, (bbox[0], bbox[1]), (bbox[2], bbox[3]),
(255, 0, 0), 2)          cv2.putText(frame, f"{action}", (bbox[0], bbox[1] - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 0, 0), 2)

    # HUD        ocr_state = "ON" if self.user_controls.is_ocr_enabled()
else "OFF"        status_text = [          f"FPS: {results['fps']:.1f}",
f"Detections: {len(detections)}",          f"Mode: {results.get('mode',
'scene')}",          f"OCR: {ocr_state}",          f"Voice: {'ON' if not
self.tts_engine.muted else 'MUTED'}",
    ]

    y = 30        for t in
status_text:
```

```
        cv2.putText(frame, t, (10, y), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255,
255), 2)
y += 25

      out = results.get("output_text")
if out:
        cv2.putText(frame, out[:70], (10, frame.shape[0] - 20),
cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 2)

   def run(self):        if not self.camera.open():            raise RuntimeError(f"Could
not open camera {self.config['camera_index']}")

      print("Camera opened. Starting
processing...")        print("Controls:")        print("
's' - Start/Pause narration")        print("  'm' -
Mute/Unmute voice")        print("  'o' - Toggle
OCR text reading")        print("  'q' - Quit")

      time.sleep(0.5)
       try:
while True:
            ret, frame =
self.camera.read()              if not ret:
time.sleep(0.1)                continue

            results = self.process_frame(frame)

            if self.config.get("display_video", False):
              self.draw_results(frame, results)
cv2.imshow("Scene Narration System", frame)

              key = cv2.waitKey(1) & 0xFF

              if key == ord("q"):
                break

              elif key == ord("s"):
                self.user_controls.toggle_narration()
```

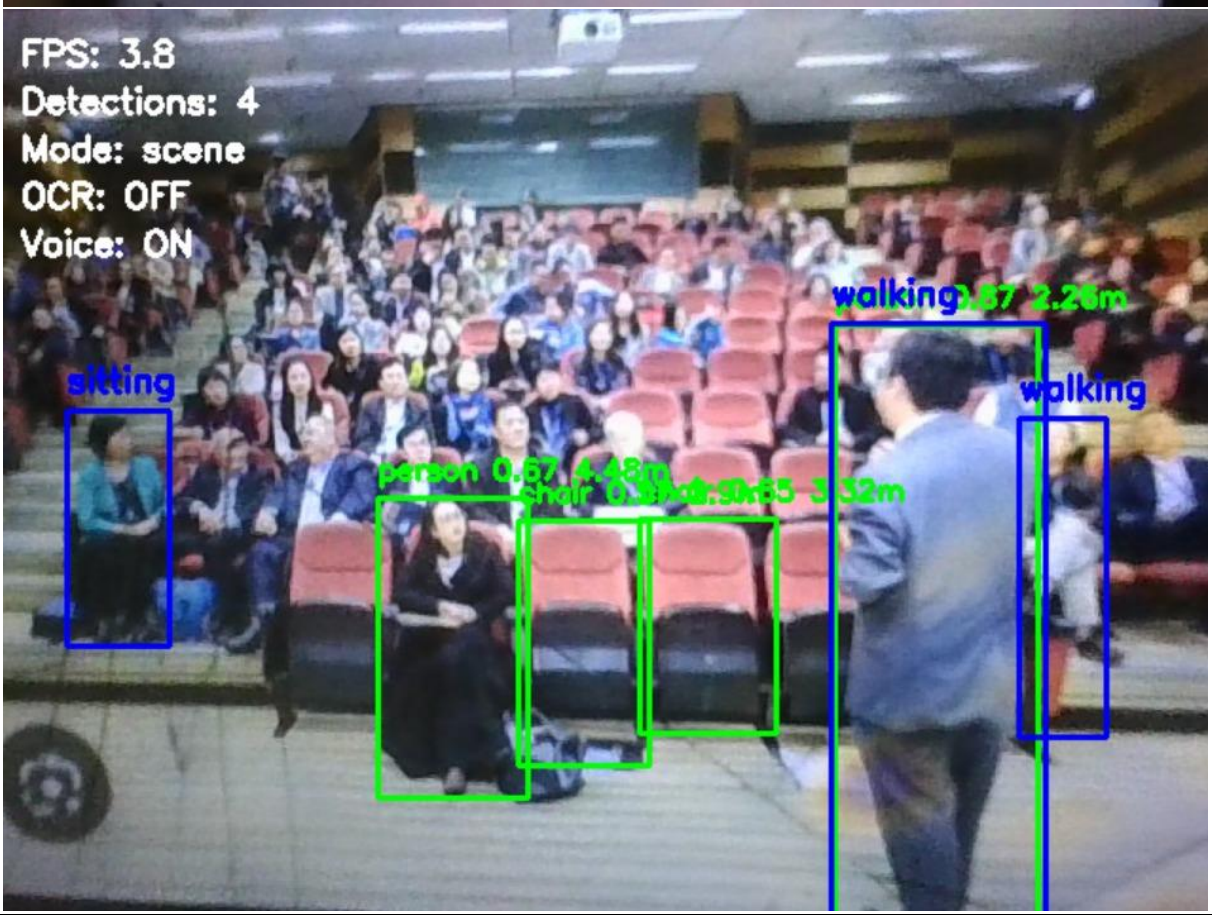
```
                self.tts_engine.speak(
                    "Narration enabled" if self.user_controls.is_narration_enabled() else
"Narration paused",
                    priority="high"
                )

            elif key == ord("m"):                    if
self.tts_engine.muted:                        self.tts_engine.unmute()
self.tts_engine.speak("Voice on", priority="high")
else:
                    self.tts_engine.mute()

            elif key == ord("o"):
                self.user_controls.toggle_ocr()

                # clear narration backlog when switching modes
self.tts_engine.stop()

                if self.user_controls.is_ocr_enabled():
                    self.tts_engine.speak("OCR mode enabled", priority="high")
else:
                    self.tts_engine.speak("OCR mode disabled", priority="high")
elif key == ord("b"):                     self.user_controls.toggle_blip()
self.tts_engine.stop()                    if self.user_controls.is_blip_enabled():
                    self.tts_engine.speak("Caption mode enabled", priority="high")
else:
                    self.tts_engine.speak("Caption mode disabled", priority="high")

            # log
            if self.frame_count % 30 == 0:
                preview = (results.get("output_text") or "None")[:60]                print(f"FPS:
{results['fps']:.1f} | Mode: {results['mode']} | Output: {preview}")

        except KeyboardInterrupt:
            print("\nStopping...")

finally:
            self.camera.release()
self.tts_engine.stop()
```

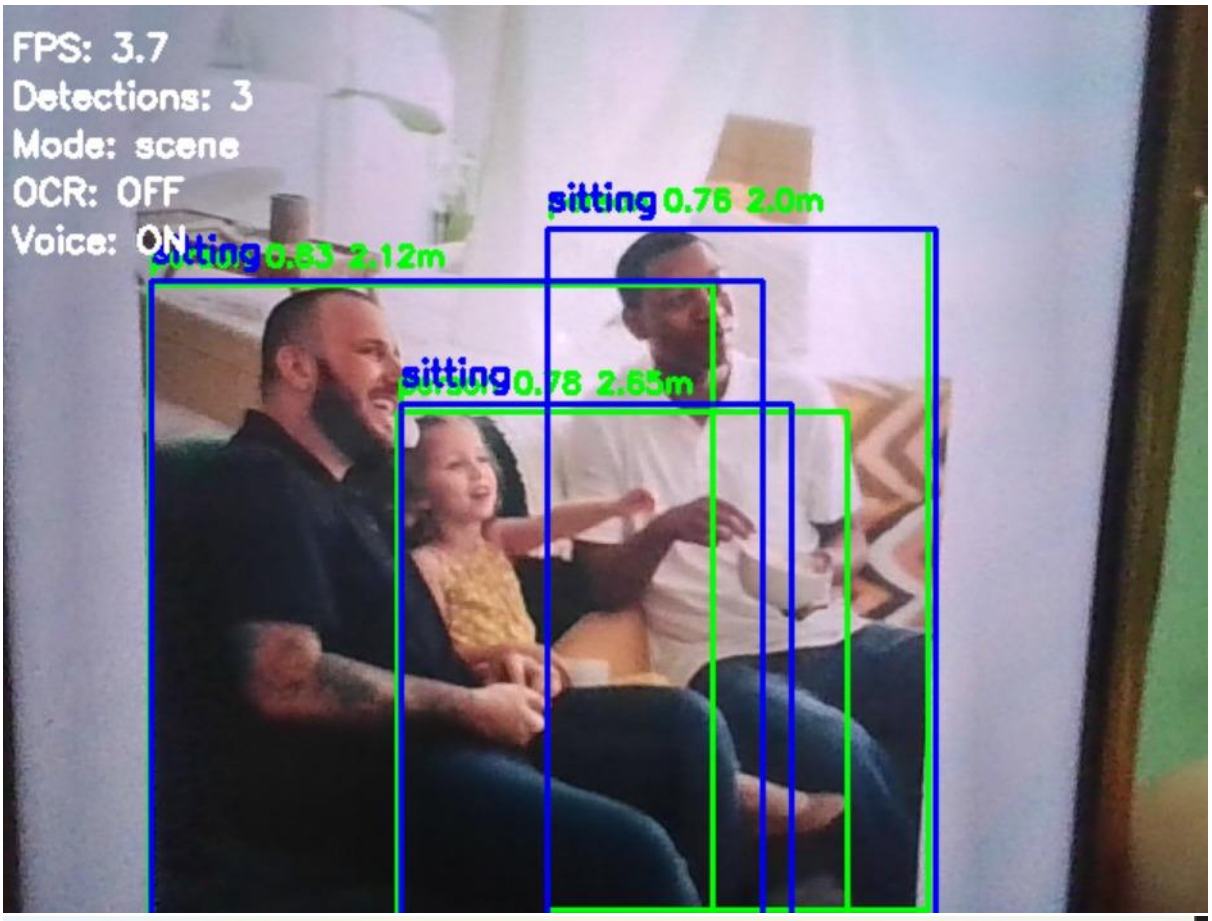
```
        # only call shutdown if it exists (your earlier error)
if hasattr(self.tts_engine, "shutdown"):
            self.tts_engine.shutdown()

        if self.config.get("display_video", False):
            cv2.destroyAllWindows()
print("System stopped.")
```

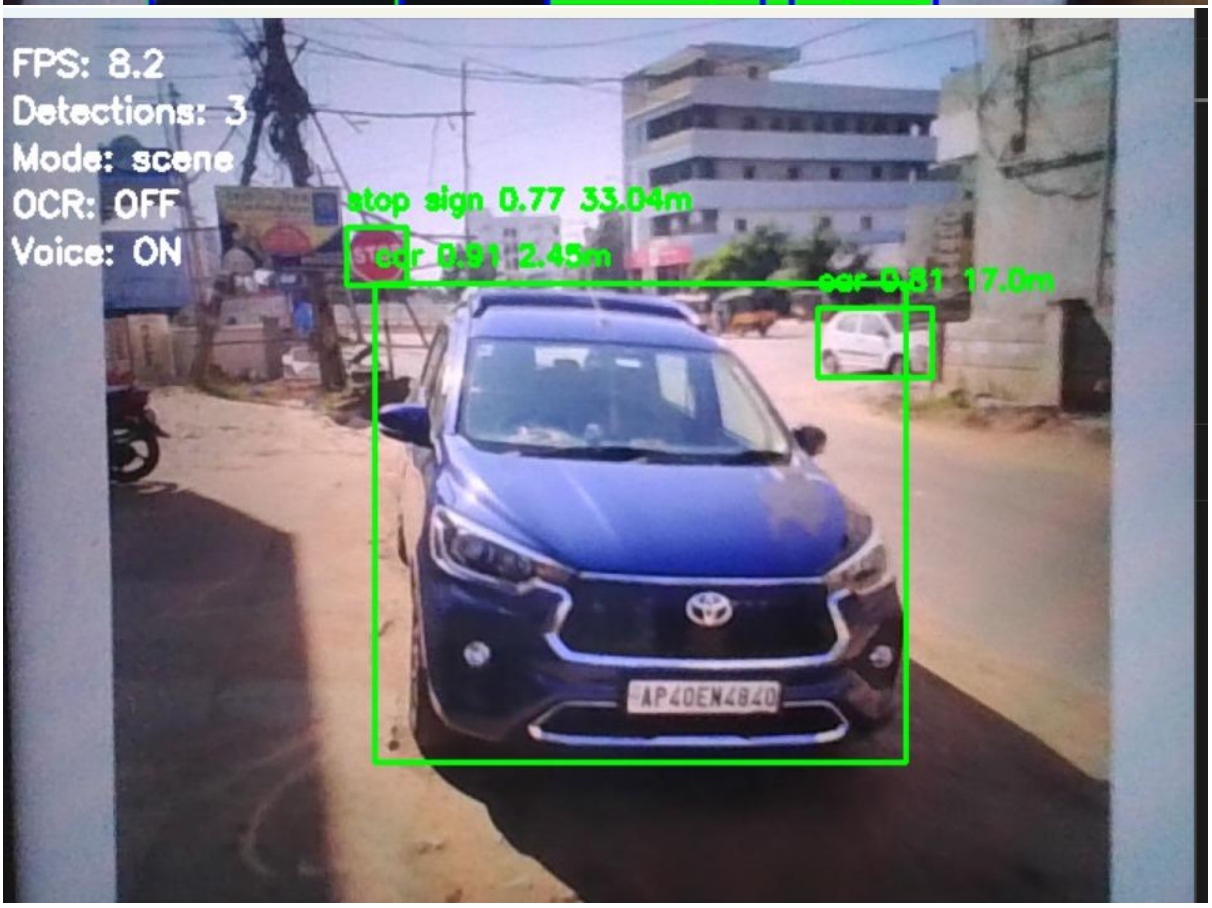
outputs

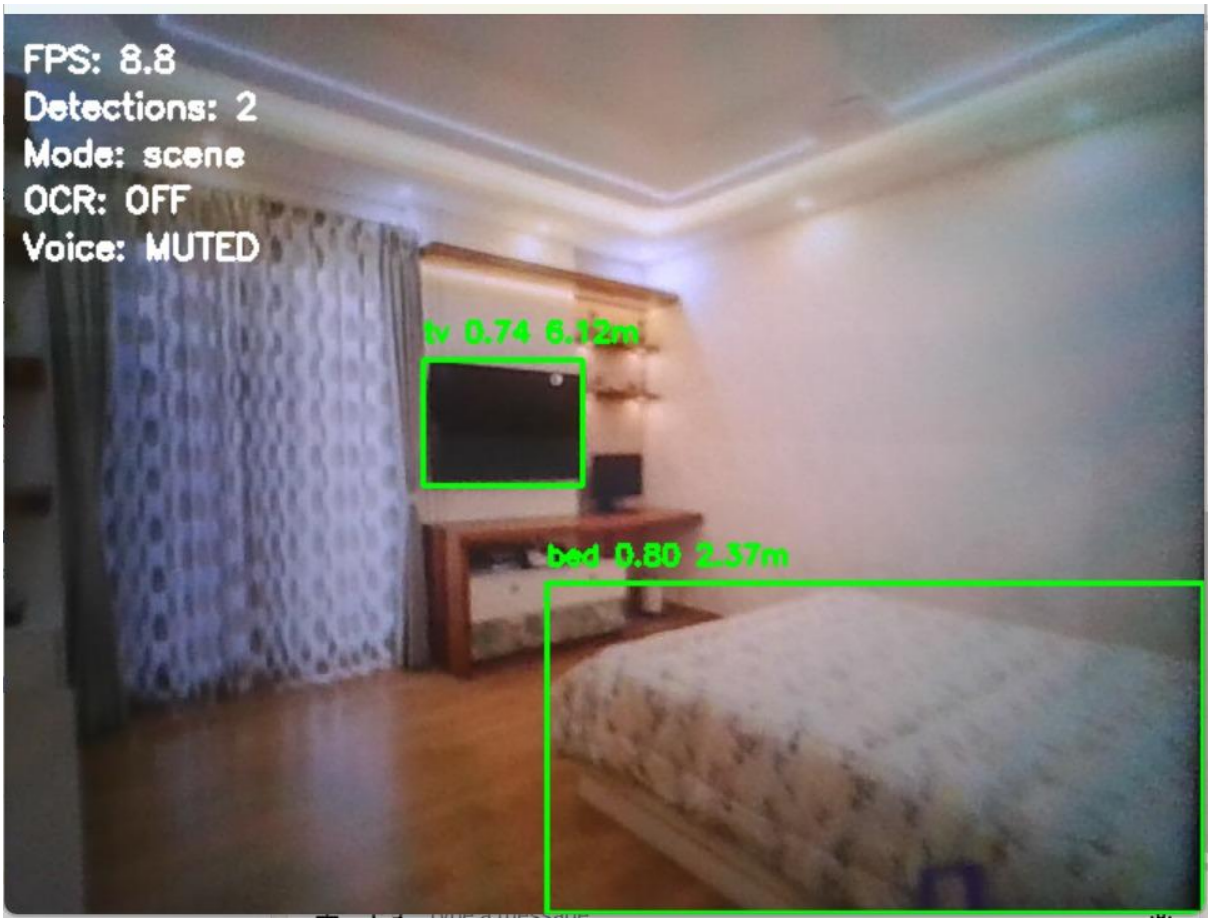
```
Controls:
  's' - Start/Pause narration
  'm' - Mute/Unmute voice
  'o' - Toggle OCR text reading
  'q' - Quit
Speak: Path ahead is clear.
Speak: Caution. person ahead, 1.7 meters.
Speak: Path ahead is clear.
Speak: Caution. person ahead, 1.7 meters.
FPS: 2.1 | Mode: scene | Output: None
Speak: Path ahead is clear.
Speak: Caution. person ahead, 1.7 meters.
Speak: Caution. person ahead, 1.6 meters.
FPS: 7.6 | Mode: scene | Output: None
Speak: Caution. person ahead, 1.7 meters.
[Controls] BLIP ON
Speak: Caption mode enabled
Speak: Scene: a man with a beard
Speak: Scene: a man with a beard and a black shirt
FPS: 1.8 | Mode: blip | Output: None
Speak: Scene: a man with a beard
FPS: 4.5 | Mode: blip | Output: None
Speak: Scene: a man is standing in front of a mirror
Speak: Scene: a man with a beard
Speak: Scene: a man holding a yellow object in his hand
FPS: 4.0 | Mode: blip | Output: Scene: a man holding a yellow object in his hand
Speak: Scene: a man holding a yellow apple in his hand
Speak: Scene: a man holding a banana in his hand
FPS: 4.6 | Mode: blip | Output: None
Speak: Scene: a man holding a yellow apple in his hand
Speak: Scene: a man holding a yellow smiley face in front of his face
Speak: Scene: a man holding a smiley face shaped object
```

FPS: 3.2
Vishopper
Detections: 4
Mode: scene
OCR: OFF
Voice: ON
walking
sitting 0.89 2.72m
sitting


FPS: 3.8
Detections: 4
Mode: scene
OCR: OFF
Voice: ON
walking
sitting
walking
person


FPS: 3.7
Detections: 3
Mode: scene
OCR: OFF
Voice: ON
sitting 0.76 2.0m
sitting 0.83 2.12m
sitting 0.78 2.65m


FPS: 8.2
Detections: 3
Mode: scene
OCR: OFF
Voice: ON
stop sign 0.77 33.04m
car 0.91 2.45m
car 0.81 17.0m
AP40EN4840




```
  'm' - Mute/Unmute voice
  'o' - Toggle OCR text reading
  'q' - Quit
Speak: Path ahead is clear.
[Controls] OCR ON
Speak: OCR mode enabled
[OCR] blocks: 0
FPS: 2.1 | Mode: ocr | Output: None
[OCR] blocks: 0
C:\Users\HP\AppData\Roaming\Python\Python313\site-packages\torch\utils\data\dataloader.py:666: UserWarning: 'pin_memory' argumen
hen device pinned memory won't be used.
  warnings.warn(warn_msg)
[OCR] blocks: 2
FPS: 5.8 | Mode: ocr | Output: None
[OCR] blocks: 2
[OCR] blocks: 2
FPS: 5.4 | Mode: ocr | Output: None
[OCR] blocks: 0
[OCR] blocks: 1
Speak: Text says: Kalyani
FPS: 5.9 | Mode: ocr | Output: None
[OCR] blocks: 1
[OCR] blocks: 0
FPS: 6.0 | Mode: ocr | Output: None
[OCR] blocks: 1
[OCR] blocks: 1
Speak: Text says: CAUTTON:|
FPS: 5.5 | Mode: ocr | Output: None
[OCR] blocks: 1
[OCR] blocks: 0

Stopping...
System stopped.
```

CONCLUSION

This project successfully demonstrates an intelligent assistive system aimed at empowering blind and visually impaired individuals by providing real-time narration of their surroundings. By integrating advanced computer vision techniques such as YOLOv11 object detection and yolov11n-pose pose estimation with voice input and output modules, the system enables dynamic recognition of objects and human actions. This transforms visual information into clear and descriptive voice feedback, enhancing the autonomy, safety, and confidence of blind users in everyday environments.

The methodology combining deep learning models and rule-based heuristics provides accurate scene understanding while maintaining real-time performance on modest hardware. Utilizing publicly available datasets like COCO ensures robust object detection across diverse scenarios. Overall, this system represents a practical and impactful application of AI for social good, contributing to digital accessibility and inclusive technology.

Future extensions may include more sophisticated action recognition, integration with GPS for navigation support, and wearable device development to broaden usability. This project lays a strong foundation for continuous innovation in assistive technology, advancing independence and quality of life for visually impaired persons.

REFERENCES

[1] Shandu, Nkosinathi Emmanuel, et al. "AI based pilot system for visually impaired people." *2020 international conference on artificial intelligence, big data, computing and data communication systems (icABCD)*. IEEE, 2020.

[2] Felix, Shubham Melvin, Sumer Kumar, and A. Veeramuthu. "A smart personal AI assistant for visually impaired people." *2018 2nd international conference on trends in electronics and informatics (ICOEI)*. IEEE, 2018.

[3] Joshi, Rakesh Chandra, et al. "AI-SenseVision: a low-cost artificial-intelligence-based robust and real-time assistance for visually impaired people." *IEEE Transactions on HumanMachine Systems* 54.3 (2024): 325-336.

[4] Deriba, Fitsum Gizachew, Betelhem Zewdu Wubineh, and Mengistu Zenebe Teka. "Review of AI technologies for enhancing the lives of visually impaired individuals: Applications, outcomes, and future directions." *2024 International Conference on Information and Communication Technology for Development for Africa (ICT4DA)*. IEEE, 2024.

[5] Shashanka, R., et al. "VisioGuide: An AI for Visually Impaired." *2025 4th International Conference on Sentiment Analysis and Deep Learning (ICSADL)*. IEEE, 2025.

[6] Lima, Rui, et al. "Visually impaired people positioning assistance system using artificial intelligence." *IEEE Sensors Journal* 23.7 (2023): 7758-7765.

[7] Khan, Muiz Ahmed, et al. "An AI-based visual aid with integrated reading assistant for the completely blind." *IEEE Transactions on Human-Machine Systems* 50.6 (2020): 507517.

[8] Thakur, Rijul, et al. "AI based visual impairment system for blind and people with least reading capabilities." *2024 IEEE Students Conference on Engineering and Systems (SCES)*. IEEE, 2024.